

LAB ASSIGNMENT 4

Server Side Web Development with Node.js(CSE 3192)

Semester: 6th

1. Implement the concept of Dependency Injection in JavaScript by implementing a user management system where a Database class handles user storage operations such as saving users, retrieving all users, and finding users by name. Use constructor injection in a UserService class to interact with the Database and perform user creation, listing, and retrieval operations, ensuring that dependencies are injected rather than directly instantiated.
2. (I) Implement the effectiveness of using the Builder Design Pattern in JavaScript for constructing complex objects such as a Boat with attributes like hasMotor, motorCount, and hullColor. Develop a Boat class and a BoatBuilder class that supports method chaining for configuring object properties, and implement a build() method to generate the final Boat object. Assess how the Builder Pattern improves code readability, flexibility, and maintainability compared to direct object construction.
(II) Implement the use of the Proxy Design Pattern in JavaScript by creating a proxy for the console object that enhances the log() function to prepend the current timestamp to every log message. Demonstrate the implementation with suitable examples and counterexamples, and examine how the proxy modifies the behavior of the original logging mechanism without changing the underlying console object.
(IV) Design and implement a Node.js program using the child_process module to demonstrate external process creation and communication. Create a parent.js file that uses fork() to spawn a child process worker.js and exchange messages between them.
3. Implement a custom asynchronous version of Array.map() in JavaScript for processing iterable data using Promises and callbacks. Create a function mapAsync(iterable, callback, concurrency, cb) that performs parallel asynchronous execution while maintaining a concurrency limit of 5 active tasks at a time. The function should preserve the order of results and invoke the final callback cb(err, results) after all operations are completed. Rewrite the program to implement a Sequential Async Map using the function mapSequential(iterable, callback, cb), where asynchronous tasks execute strictly one after another in ordered execution. Demonstrate both implementations using an asynchronous callback function that returns the square of a number after a delay. Compare the behavior of parallel execution with concurrency control and sequential execution in terms of execution order, performance, memory usage, and practical applications such as API request handling, task scheduling, and file processing.
4. (I) Implement a generator function taskFlow() that yields multiple asynchronous tasks and controls their execution flow using JavaScript generators and Promises.

(II) Write a non-blocking JavaScript function to calculate the sum of numbers from 0 to n using asynchronous chunk processing. Divide the computation into smaller chunks and process them asynchronously to avoid blocking the event loop.

(III) Write a program to implement the concept of Inversion of Control (IoC) using a suitable Node.js program and demonstrate how dependency handling can be delegated to external modules or callback functions.

(IV) Design and evaluate a user-defined function `findRegex()` in Node.js that asynchronously reads multiple files and searches for a specified regular expression pattern using the `EventEmitter` concept. Analyze how event-driven programming improves asynchronous file handling and pattern-matching efficiency in Node.js applications.

(V) Implement a Node.js program to perform recursive parallel file searching using the function `searchFilesAsync(path, text, maxLimit, done)` with a concurrency limit of three simultaneous file reads. Also develop the function `orderedPromiseRunner(jobs, callback)` to execute promises sequentially while preserving the order of execution.

5. Design and implement a `parallelSearch(dir, keyword, limit, cb)` function in Node.js to perform recursive parallel file searching with concurrency control, limiting simultaneous file reads to three. Implement another function `sequentialAll(tasks, cb)` using `Promise.all()` logic to execute asynchronous tasks sequentially while maintaining the order of execution. Analyze and compare parallel execution with concurrency control and sequential promise execution in terms of performance, resource management, and execution flow.
6. (I) Write a program to examine the implementation of the Middleware Pattern in Node.js using a `Middleware` class that manages function pipelines through `use()` and `run()` methods. Analyze how the pattern enables sequential task execution, modular processing, and flexible request handling in server-side applications.
(ii) Write a program in node js to implement the Strategy Design Pattern in JavaScript using a `Config` class that stores common data and applies different saving strategies. Analyze how `JSONStrategy` and `TextStrategy` improve flexibility, reusability, and maintainability by enabling data to be saved in JSON and plain text formats dynamically.
(II) Implementation the concept of Decorator Design Pattern in Node.js for dynamically adding new behavior to an object without changing its original class. Analyze how the decorator pattern supports flexibility, extensibility, and modular application development.
(III) Design and evaluate the implementation of the Proxy Design Pattern in a stack-based calculator system using `StackCalculator` and `SafeCalculatorProxy` classes. Develop a proxy mechanism that controls access to calculator operations by validating actions such as multiplication on the top two stack values before execution. Analyze how the proxy pattern improves security, controlled access, reliability, and maintainability in object-oriented application design. Afterwards apply the following sequence of operations: insert number 99, insert number 5, multiply both and display the result.

(IV) Write a program to analyze the concept of the Singleton Design Pattern used in Node.js with a suitable program. Evaluate how the singleton pattern ensures that only one instance of an object is created and examine its advantages in managing shared resources, configuration settings, and application-wide services.

(V) Write a program to create a primary process that forks two worker processes, where every worker should run an HTTP server on port 5000.

7. (I) Write a program to implement a Factory Pattern in Node.js for creating user objects with properties such as name, age, and role. Analyze how the Factory Pattern improves abstraction, scalability, and maintainability in object-oriented application development.
8. Develop a function `combineDistinctLines()` that merges multiple arrays of text lines and returns only the non-duplicate lines. Also, implement a callback-based function `mergeTextFiles(file1, file2, file3, ..., outputFile, callback)` that combines the contents of three or more source files into a single destination file. Implement the `mergeTextFiles()` function to read multiple source files asynchronously from the filesystem, split their contents into separate lines, combine the lines while eliminating duplicates using `combineDistinctLines()`, and write the final merged output into the destination file.
9. Write a program for implementation of graceful worker restarting in Node.js when the `SIGUSR2` signal is received. Develop a mechanism where old workers terminate only after new workers begin listening, and simulate delayed request handling by displaying the worker process ID in the response.
10. Implement the concept of a peer-to-peer load balancing system in Node.js by implementing a program that distributes incoming requests among multiple peer nodes. Demonstrate how load balancing can improve resource utilization and system performance in a distributed environment.